

A Cascaded Broad Learning Network Embedded Image Features for Malware Traffic Classification

Junzhi Xu, Yibin Zhang^{1b}, *Graduate Student Member, IEEE*, Kaijie Zhou, Qin Wang^{2b}, *Senior Member, IEEE*, Minyu Hua, Lin Shan^{3b}, *Senior Member, IEEE*, Yun Lin^{4b}, *Senior Member, IEEE*, and Guan Gui^{5b}, *Fellow, IEEE*

Abstract—Malware traffic classification (MTC) is a critical technology in network security, Internet of Things (IoT) devices, and big data traffic, and has been mainly used for encrypted malicious traffic detection and various network traffic classification. The traditional MTC technology mainly relies on payload-based deep packet inspection methods and port-based identification detection techniques. Meanwhile, In recent years, many studies have begun to explore the application of machine learning (ML) and deep learning (DL) in MTC tasks. However, ML methods require extracting excellent feature information, which can be challenging for traffic detection tasks. DL methods require excellent computing power equipment, such as GPUs, but also have the drawbacks of long training time and poor interpretability. Consequently, this paper proposes an image feature embedding cascaded broad learning network (IFCBLN) to address the shortcomings of ML and DL in MTC tasks. The proposed IFCBLN method utilizes multiple cascaded broad learning networks (BLN) combined with image feature extractors to improve the original BLN. Meanwhile, it achieves good performance in MTC tasks. In the datasets of USTC-TFC2016 and ISCXPVN2016, the proposed IFCBLN achieves better performance than ML and DL methods. The best results of the algorithm achieve an accuracy of 99.543% and 90.93%, respectively. The proposed method reduces computational overhead by 98% compared to the DL method.

Index Terms—Malware traffic classification, image feature embedding cascaded broad learning, feature extraction, network security.

I. INTRODUCTION

MALWARE traffic classification (MTC) [1], [2], [3], [4] technology is an excellent way to detect encrypted

malicious traffic for relevant network management departments and device providers. With the development of the 6th generation mobile networks [5], [6] technology and Internet of Things (IoT) [7], [8], [9], [10] technology, data transmission speed is getting faster and the amount of data is getting larger. Network attacks [11] are also increasing and becoming more covert. The development of IoT technology has enabled traditional electronic devices to be connected to the network, bringing great convenience to public lives. For example, smart homes, smart cities, autonomous driving, etc. However, while bringing convenience, these devices are more susceptible to malicious traffic attacks. Illegal traffic can steal important confidential information from users and businesses. In these situations, malicious traffic detection technology has also developed and achieved significant results. Traditional traffic detection techniques include deep packet inspection (DPI) [12] methods based on payloads (TCP, UDP, IP, etc.), as well as identification and detection techniques based on ports (HTTP, FTP, SNMP, etc.), which can effectively detect illegal network traffic. However, they are powerless against encrypted malicious traffic and the complex network environment. Therefore, finding more effective and robust methods to detect encrypted malicious traffic is necessary.

To solve this problem, some researchers have applied machine learning (ML) methods to malicious traffic detection [13], [14], [15], [16], [17] techniques. Traditional machine learning methods include support vector machine (SVM) [18], naive Bayes, and random forest algorithms. ML has significant advantages in training time, but it requires extracting sufficient and effective feature information. Therefore, researchers need to expend a lot of energy in the feature extraction process. With the development of computing power in computer devices, deep learning (DL) [19], [20], [21], [22], [23] has also emerged. DL has also been applied in traffic detection tasks. Common DL networks include convolutional neural network (CNN), gated recurrent unit (GRU), and long short-term memory (LSTM). Zhao et al. [24] proposed a CNN-based CL-ETC malicious traffic detection method. Yao et al. [21] proposed a HAN traffic classification method based on LSTM. Wang et al. [25] proposed deep learning-based methods for real-valued convolution neural networks (RVCNN) and real-valued convolution neural networks (CVCNN). DL technology can automatically extract potential feature information without the need for manual feature extraction. However, deep learning requires powerful computing devices, such as GPU, which can provide acceleration. Although GPU can be used, the inference

Received 19 June 2024; revised 31 October 2024; accepted 16 December 2024. Date of publication 23 December 2024; date of current version 8 August 2025. This work was supported in part by the Natural Science Foundation of China under Grant 62471247, and in part by the Key Project of the Natural Science Foundation of the Higher Education Institutions of Jiangsu Province under Grant 22KJA510002. The associate editor coordinating the review of this article and approving it for publication was N. Zhang. (Corresponding author: Guan Gui.)

Junzhi Xu, Yibin Zhang, Qin Wang, Minyu Hua, and Guan Gui are with the College of Telecommunications and Information Engineering, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: 1023010410@njupt.edu.cn; 2021010208@njupt.edu.cn; wangqin@njupt.edu.cn; 1022010410@njupt.edu.cn; guiguan@njupt.edu.cn).

Kaijie Zhou is with the School of Physical and Electrical Engineering, Huaiyin Normal University, Huaian 223300, China (e-mail: zhoukaijie@hytc.edu.cn).

Lin Shan is with the Center for Information Infrastructure, Shinshu University, Nagano 390-8621, Japan (e-mail: shanlin@shinshu-u.ac.jp).

Yun Lin is with the College of Information and Communication Engineering, Harbin Engineering University, Harbin 150000, China (e-mail: linyun@hrbeu.edu.cn).

Digital Object Identifier 10.1109/TCCN.2024.3522143

time is still extremely long. Therefore, this article proposes a MTC method based on broad learning networks (BLNs).

BLN [26], [27], [28], [29], [30], [31], [32], [33] is a neural network structure with only one layer, consisting of a forward propagation network without a feedback mechanism to automatically adjust weights. Although there is only one layer of a network, the network width can be infinitely expanded. The weight coefficient is randomly generated in BLN. For newly added nodes, only the pseudo inverse of the newly added node needs to be calculated. The training time is greatly reduced. However, the feedback mechanism and convolution calculation of DL networks greatly increase time overhead. Consequently, widening the network width does not significantly increase the training time of the network compared to DL networks. Due to its unique results, it has a fast training speed and can achieve good experimental performance. BLN has demonstrated excellent performance in multiple application fields. Firstly, it has achieved good performances in the field of specific emitter identification (SEI), and the signal feature embedded broad learning network (SFEBLN) method proposed by Zhang et al. [34] has been applied to the SEI field. This method not only has fast training speed but also recognition accuracy comparable to deep learning algorithms. Secondly, it also excels in the field of industrial manufacturing. The ABTCI method proposed by Liu et al. [35] is applied in industrial production environments, and BLN can effectively monitor the operation of machine rotors and large equipment in industrial workshops. It provides a good method for the automation and unmanned operation of factory workshops. In addition, it has also achieved success in tasks such as computer vision (CV) [36], as it can process large-scale image data and extract important features. It is also widely applied in automatic modulation classification [38], [39]. Overall, BLN has shown excellent application potential in processing large-scale, high-dimensional data, especially in SEI, industrial manufacturing, and CV fields.

Given the excellent algorithm performance of BLNs, we propose an image feature embedding cascaded broad learning network (IFCBLN) method. In this paper, we introduce classic image processing methods into malicious traffic detection, effectively avoiding the complex detection methods of traditional malicious traffic techniques. Converting text data into image format data, and extracting feature [37] information from the image using image processing methods. Next, we cascade the BLN in different ways and evaluate the algorithm performance of different cascaded networks. Then, based on the above situation, a new network traffic classifier is applied to the network traffic classification task. The main contributions of this paper can be briefly summarized as follows:

- We propose an image feature embedding cascaded broad learning network (IFCBLN) method. The proposed method uses image processing methods to process transformed data and cascaded BLN for malicious traffic detection. The IFCBLN method is trained on the central processing unit (CPU).
- In contrast to traditional MTC methods, we innovatively design image feature nodes, which extract image

features from network traffic through image processing techniques. It is embedded into the proposed algorithms. Additionally, we implement three different cascades for the feature mapping nodes and enhancement nodes of basic BLN. The improved algorithms demonstrate excellent performance in MTC tasks.

- The proposed IFCBLN method was experimented with in the public datasets of USTC-TFC2016 and ISCXVPN2016. Compared with two ML algorithms and three DL algorithms, the proposed algorithm can effectively learn robust features and achieve better performance. At the same time, it ensures that time consumption is much less than that of DL algorithms.

The remainder of this paper is organized as follows. In Section II, we introduce the recent research progress in the field of MTC. In Section III, we introduce the system model and the mathematical model for MTC problems. In Section IV, We introduce the technical details of the proposed IFCBLN method, including the basic network, improved network architecture, and algorithm flow chart. The experimental results and analysis are given in Section V. Finally, Section VI concludes the research work of this paper.

II. RELATED WORKS

A. Traditional ML-Based MTC Methods

As encryption technologies advance and new applications emerge, traditional methods based on port numbers and Deep Packet Inspection (DPI) have become inadequate for handling the complexities of modern network environments. Yamansavascular et al. [40] extract 111 features from the UNB ISCX dataset and achieved 93.94% classification accuracy using a KNN classifier on 14 applications. Similarly, Koroniotis et al. [41] extract 46 features from captured traffic and stored them in CSV format, achieving nearly 99% accuracy with an SVM classifier. Chebrolu et al. [42] employ two feature selection methods—Bayesian Networks and Regression Trees—to identify key traffic features. However, traditional ML approaches, which rely heavily on manually designed features, face challenges when dealing with large and complex network traffic, often leading to difficulties in feature design and limited generalization capabilities.

B. DL-Based MTC Methods

With the rising complexity of modern cyberattacks [43], [44], [45], [46], [47], MTC has become a critical focus in cybersecurity. DL techniques have gained prominence due to their ability to extract meaningful features from raw network data. However, various challenges persist, including limited datasets, computational constraints, and the need for models that can efficiently adapt to new threats. This section consolidates the contributions and outcomes of recent DL-based MTC research [48], [49], [50], [51], [52], [53], presenting how each approach tackles key challenges in the field. Zhang et al. [48] propose a few-shot malware traffic classification (FS-MTC) framework that addresses the data scarcity problem in MTC. Their method combines knowledge transfer and neural architecture search (NAS) to develop an

adaptive model that performs well with minimal training samples and limited computational resources. The experimental results demonstrate the effectiveness of this method, with the USTC-TFC2016 dataset achieving 86.91% accuracy in 1-shot scenarios, outperforming traditional approaches that struggle with data scarcity. Ge et al. [50] propose MetaCluster, an interpretable classification framework that enhances the transparency and reliability of DL-based MTC models. MetaCluster uses semantic prototypes to generate meaningful feature representations at multiple granularities. This method ensures robust classification while reducing model overfitting and computational overhead. Compared to other DL models, MetaCluster reduces parameter consumption by 79.52% to 91.78%, improves operational speed by 71.37%, and maintains stable or slightly improved F1 scores.

Additionally, it offers interpretability by visually representing the importance of image, text, and statistical features, bridging the gap between the complexity of DL and practical cybersecurity needs. Sheng et al. [51] introduce Scanner-Hunter, a novel system for identifying malicious scanning groups targeting industrial control systems (ICS) by leveraging honeynet data. Scanner-Hunter employs a self-expanding multi-class classification (SEMCC) model and IP prefix judgment to accurately identify sophisticated scanning patterns. The model effectively analyzes six years of honeynet data, uncovering previously unknown scanning groups, some with advanced persistent threat (APT) characteristics. The ability of a system to identify suspicious attackers enhances proactive threat defense by providing actionable intelligence on potential cyberattack campaigns. Given the resource limitations of IoT devices, He et al. [53] propose a lightweight intrusion detection method based on feature grouping and protocol parsing. This approach focuses on reducing computational overhead without compromising detection accuracy. The method achieves 99.5% classification accuracy across three IoT datasets, significantly outperforming traditional methods in terms of computational efficiency. Its lightweight design makes it ideal for deployment in resource-constrained environments, ensuring robust IoT security.

The reviewed studies demonstrate diverse applications of DL techniques in MTC, addressing various challenges such as data scarcity, model interpretability, catastrophic forgetting, and computational constraints. These advancements illustrate the growing role of DL in addressing the evolving landscape of cyber threats. Future research can benefit from integrating these approaches to develop more adaptive, interpretable, and efficient MTC systems.

C. BL-Based MTC Methods

In recent years, some researchers have introduced Broad Learning (BL) into the field of malware traffic detection and achieved promising results. BLN offers several advantages, making them well-suited for tasks like malware traffic classification. One key benefit is their ability to rapidly learn features without requiring extensive training time, unlike DL models that often demand large datasets and high computational resources. BLN models are also more interpretable,

providing clearer insights into the learned features and decision processes, which is particularly useful in cybersecurity. Additionally, their lightweight architecture allows them to achieve competitive performance. Lin et al. [54] tackle the challenge of imbalanced network traffic classification by introducing MC-OCBLS (Maximum Correntropy-based One-Class Broad Learning System). Their method enhances the robustness of the model by using the maximum correntropy criterion (MCC) to handle noise and outliers effectively. To improve performance further, they propose SMC-OCBLS, an ensemble boosting model that increases diversity by disrupting feature columns during training, forming multiple feature spaces. This strategy enables the ensemble to train hybrid classifiers across varied feature spaces, improving accuracy and generalization. Experimental results show that MC-OCBLS and SMC-OCBLS outperform existing advanced methods, particularly in handling noisy and imbalanced network traffic datasets. Zhang et al. [1] focus on lightweight MTC for IoT environments, aiming to address the limitations of traditional ML and DL techniques. Quickly train models by adding feature layers horizontally rather than increasing model depth. Minimize computational overhead without sacrificing accuracy. The model showcases a significant reduction in computational cost, making it highly suitable for deployment in IoT environments.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Model

Network traffic is transmitted in the data link and routing ports, which are added to hardware collection devices to collect network traffic data. Refining the data using software analysis tools and converting it into message-format data.

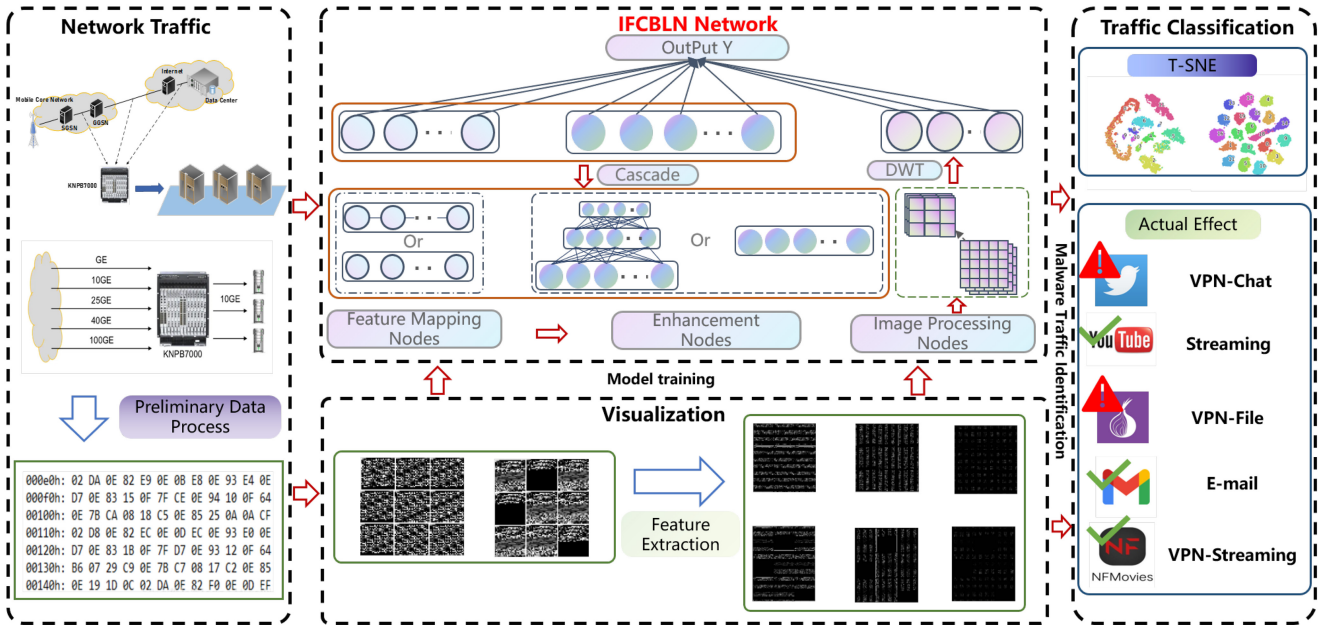
This paper uses the IFCBLN to complete the task of network traffic classification. The model is shown in Fig. 1. The four parts of the traffic recognition system include 1) Network Traffic Collection, 2) Visualization, 3) IFCBLN, and 4) Traffic Classification. Firstly, it is necessary to convert the collected message format data into image-type data. Then, the converted data is placed into the IFCBLN model, which includes feature mapping nodes, enhancement nodes, and image processing nodes. Finally, IFCBLN is capable of identifying various types of network traffic and encrypted malicious traffic. The details of the model architecture will be explained in detail in the following texts. The proposed model can effectively solve the MTC problem in this paper.

B. Problem Formulation

This paper finds a fast and effective method to solve the MTC problem. We use encrypted malicious traffic and normal traffic as the overall input. And convert the input data format. It can be expressed as follows:

$$\hat{x}_j(t) = \mathcal{T}[x_j(t)] \quad (1)$$

where $\hat{x}_j(t)$ is the data after format conversion, $\mathcal{T}$ means data format transformation, $x_j(t)$ is network traffic header format data.



Essentially, the proposed MTC method solves the problem of matching traffic packets with network traffic types in real life. Encrypting malicious traffic is similar to it. As long as the type of malicious traffic can be identified, specific measures can be taken against illegal network traffic. Therefore, traffic packets and network traffic types can be summarized as datasets $\mathbb{D}\{\hat{x}_j, y_j\}_{j=1}^N$, where y_j are the real network traffic labels. It is the label corresponding to the real network traffic type. The MTC task of this article can be summarized as follows:

$$\hat{y}_j = f_{MTC}(\hat{x}_j(t), W) \quad (2)$$

where f_{MTC} stands for the mapping function used to identify various network traffic data and $\hat{y}_j$ are the network traffic labels of prediction. W is the weight coefficient after iteration.

To evaluate the effectiveness of the proposed MTC model. We will compare the predicted network traffic type labels with the actual network traffic type labels. The probability of this series of data can be defined as accuracy as follows:

$$Acc = \left\{ \sum_{j=1}^M (\hat{y}_j = y_j) \right\} / M \quad (3)$$

and the corresponding error rate is $Error = 1 - Acc$. At the same time, we also need to optimize the function during the training process to improve the accuracy of the model and reduce its error rate, which can be summarized as follows:

$$W = \arg \min_{\mathcal{W}} \epsilon\{\hat{y}_j = f_{MTC}(x_j, \mathcal{W}), y_j\} \quad (4)$$

where ϵ stands for the difference between real labels y_j and predicted labels $\hat{y}_j$, which is the error rate *Error* and $\mathcal{W}$ is the weight coefficients that cause the predicted labels to deviate from the true labels.

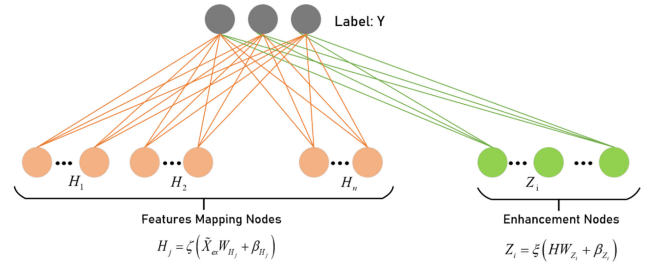


Fig. 2. Illustration of basic BLN structure.

IV. THE PROPOSED MTC METHOD USING IFCBLN

In this section, the proposed IFCBLN algorithm for MTC problems is introduced, which includes features mapping nodes, enhancement nodes, and image processing nodes, as well as three cascading methods of features mapping nodes and enhancement nodes. Before that, we will first introduce the preliminaries of the classic BLN and then provide a detailed introduction to the proposed IFCBLN method. Finally, the flowchart and summary of the algorithm are introduced in this section.

A. Theoretical Foundations

The basic BLN structure can be described as Fig. 2. The model structure only has a single layer of network, which has a forward propagation network and has no feedback mechanism. The feedback mechanism has been removed, allowing the network to train at a fast speed. At the same time, with its unique network structure, it not only has fast speed but also excellent performance. Next, we will introduce the main components of basic BLN. Three parts are as follows:

1) *Features Mapping Nodes*: The features mapping nodes $\mathbf{H}$ are obtained by linear transformation of input data $\mathbf{X}$. In

the BLN, there can be multiple feature mapping nodes, which can be represented as follows:

$$\mathbf{H} = [\mathbf{H}_1, \mathbf{H}_2, \dots, \mathbf{H}_n] \quad (5)$$

where $\mathbf{H}_n$ is the part of the feature mapping node, and the linear transformation of the input data is as follows:

$$\mathbf{H}_j = \zeta(\tilde{\mathbf{X}}_{ex} \mathbf{W}_{H_j} + \beta_{H_j}) \quad (6)$$

where ζ represents the linear transformation of the BLN, and $\tilde{\mathbf{X}}_{ex}$ is the preprocessed data, which is processed differently in different application fields. $\mathbf{W}_{H_j}$ is a random coefficient matrix generated by the system and follows a Gaussian distribution. β_{H_j} is a sparse representation of the matrix after linear transformation. The overall processing flow of the above formula is as follows: first, preprocessing the input data and then multiplying the input coefficient matrix with the random coefficient matrix to complete random convolution. Next, the obtained coefficient matrix needs to normalize the coefficients of the entire matrix through the maximum and minimum values. Finally, sparse representation is performed to reduce the correlation between feature mapping nodes. Avoid generating weights with a correlation that is too high, reduce waste, and improve the utilization of weights. From this process, it can be seen that this network structure is strongly random.

2) *Enhancement Nodes*: The enhancement nodes $\mathbf{Z}$ generated through linear transformation cannot fully complete the training task of the network structure. The features obtained from linear transformation are extremely limited. Therefore, it is necessary to supplement other feature information through other means. Nonlinear transformations, which are opposite to linear transformations, can effectively compensate for this deficiency. The enhancement nodes in this summary are obtained by non-linear transformation of the coefficient matrix obtained from the feature mapping node section. Enhancement node processing can be described as follows:

$$\mathbf{Z} = [\mathbf{Z}_1, \mathbf{Z}_2, \dots, \mathbf{Z}_m] \quad (7)$$

where $\mathbf{Z}$ is composed of m enhancement nodes windows, and $\mathbf{Z}_m$ is one of the enhancement node windows. The specific nonlinear transformation is as follows:

$$\begin{aligned} \mathbf{Z}_i &= \xi(\mathbf{H} \mathbf{W}_{z_i} + \beta_{z_i}) \\ &= \xi([\mathbf{H}_1, \mathbf{H}_2, \dots, \mathbf{H}_n] \mathbf{W}_{z_i} + \beta_{z_i}) \\ &= \xi([\dots \zeta(\tilde{\mathbf{X}}_{ex} \mathbf{W}_{H_l} + \beta_{H_l}), \dots] \mathbf{W}_{z_i} + \beta_{z_i}), \end{aligned} \quad (8)$$

where $\mathbf{H}$ represents the number of windows for the features mapping nodes, ξ is a nonlinear transformation function for the feature mapping node, and $\mathbf{W}_{z_i}$ and β_{z_i} are randomly generated weight matrices and biases corresponding to the enhancement nodes, which also follow a Gaussian distribution. The enhancement nodes processing process is as follows: firstly, all coefficient matrices $\mathbf{H}$ of the feature mapping nodes are placed into the enhancement node network as input data. Secondly, the input matrix $\mathbf{H}$ is multiplied by the randomly generated matrix $\mathbf{W}_{z_i}$ for the purpose of random convolution and bias terms. Normalizing the coefficients in the matrix after

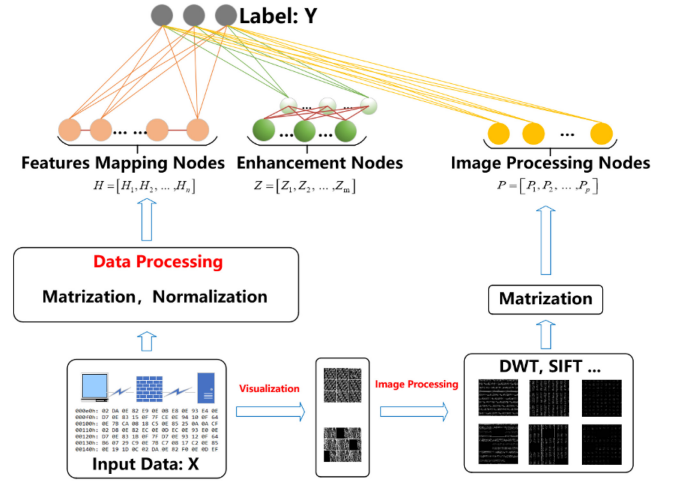


Fig. 3. Architecture of the IFCBLN method for MTC problems.

calculation. Finally, non-linear activation functions are used to perform non-linear operations on the coefficient matrix. In this way, the enhanced node processing is completed, and adding a nonlinear activation function can increase the nonlinear factors of the network.

3) *Nonlinear Activation Functions*: This summary provides a concise description of nonlinear functions. In neural networks, adding nonlinear functions is to enhance their non-linear characteristics and apply them to nonlinear problems. In reality, the vast majority of problems are nonlinear problems. The common nonlinear activation functions are as follows:

$$\tan \text{sig}(b) = \frac{2}{1 + e^{-2b}} - 1 \quad (9)$$

$$\text{sigmoid}(b) = \frac{1}{1 + e^{-b}} \quad (10)$$

$$\tanh(b) = \frac{e^b - e^{-b}}{e^b + e^{-b}}. \quad (11)$$

These activation functions are often used for both DL and BLN.

B. Our Proposed IFCBLN Method

The MTC problem is often solved by DL and its improved algorithms, so this paper adopts a broad learning network to address MTC task. In this paper, we propose an IFCBLN method for the MTC problem. The detailed model architecture of the proposed method is shown in Fig. 3. This mainly includes three cascade methods of features mapping nodes and Enhancement Nodes for BLN: a cascade of features mapping nodes (CF), a cascade of enhancement nodes (CE), and a cascade of features mapping nodes and enhancement nodes (CFE), as well as adding new incremental nodes: image processing nodes. The IFCBLN method mainly extracts the features of malicious traffic data through network cascade and the idea of extracting image features from network traffic data. The main components of IFCBLN are as follows:

1) *Image Processing Nodes*: Ordinary malicious traffic data is transformed into image format data through format transformation. Extracting features from network traffic data

in header format is too cumbersome and difficult. Therefore, converting the data format is necessary. There are relatively few methods for feature extraction from traffic data, and the research in this area remains incomplete. On the other hand, image processing is more mature, with many established methods available for feature extraction. Therefore, traffic data is converted into image data to leverage these advanced techniques. Extracting malicious traffic data feature information in image format through image processing methods. At present, researchers have conducted comprehensive research in the field of CV, and there are many methods for image processing to extract feature information. This conclusion adopts classical image processing, due to its fast processing speed and good processing results. Extract gradient changes in image format data through image processing. Significant gradient location can be helpful for network traffic classification tasks. The processing process of image processing nodes can be expressed as:

$$\mathbf{X}_{in} = p_{in}(\mathbf{X}, \Psi) \quad (12)$$

where $\mathbf{X}_{in}$ is the coefficient matrix obtained by processing the input data through image processing nodes. $\mathbf{X}$ and Ψ are the input data and the parameters that the transformation function may require. Among them, the classic image processing methods are as follows:

$$X_{in}[j, k] = \frac{1}{\sqrt{M}} \sum_k f[n] \Phi_{j,k}[n] \quad (13)$$

where $f[n]$ is the input data, $\Phi_{j,k}[n]$ is the wavelet basis function, and in discrete cases, it is a matrix. The formula is discrete wavelet transform (DWT). Common wavelet bases include Haar, Daubechies, etc. After transformation, the gradient transformation feature information of the image can be extracted. However, its output cannot be directly input into the network, and we still need to perform nonlinear transformation on it. We define image processing nodes as follows:

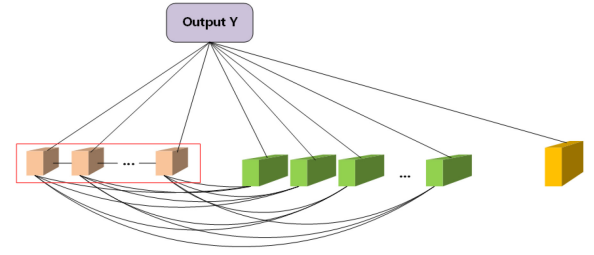
$$\mathbf{P} = [\mathbf{P}_1, \mathbf{P}_2, \dots, \mathbf{P}_p] \quad (14)$$

where $\mathbf{P}_k$ is the k -th image processing nodes window, which can be obtained through the following processing steps, as follows:

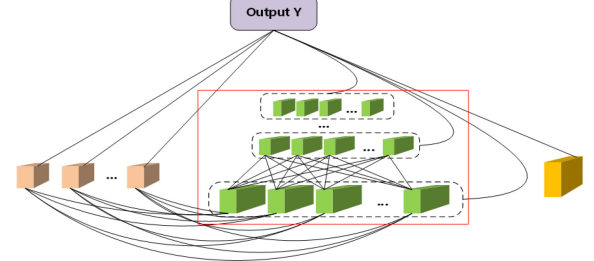
$$\mathbf{P}_k = \gamma(\mathbf{X}_{in} \mathbf{W}_{P_k} + \beta_{P_k}) \quad (15)$$

where γ is the non-linear transformation activation function of the enhancement nodes. $\mathbf{W}_{P_k}$ and β_{P_k} are the randomly generated weights for image processing nodes and the parameters that may be required for nonlinear transformations, respectively. In these nodes, we perform image processing on the converted format data. After the image processing transformation, we matrix the image format data to easily put it into the BLN. Finally, a non-linear transformation of the input data is performed to extract non-linear features. So, image processing nodes can be represented by $\mathbf{P}$.

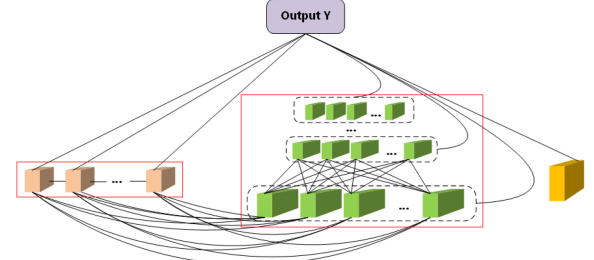
2) *Cascade of Features Mapping Nodes*: We introduce the cascaded feature mapping nodes, as shown in Fig. 4(a). We make a cascading connection for feature mapping nodes. Next,



(a) Cascade of features mapping nodes



(b) Cascade of enhancement nodes



(c) Cascade of features mapping nodes and enhancement nodes

Fig. 4. The cascaded broad learning network of features mapping nodes and enhancement nodes.

we need to create the first feature mapping nodes. The input data is placed in them, represented as follows:

$$\mathbf{H}_1 = \phi(\mathbf{XW}_{h_1} + \beta_{h_1}), \quad (16)$$

where $\mathbf{W}_{h_1}$ and β_{h_1} are randomly generated weight matrices and parameters used, respectively, and ϕ is the linear transformation. Then, the output of the first node is placed into the second feature mapping node, expressed as follows:

$$\begin{aligned} \mathbf{H}_2 &= \phi(\mathbf{H}_1 \mathbf{W}_{h_2} + \beta_{h_2}) \\ &= \phi(\phi(\mathbf{XW}_{h_1} + \beta_{h_1}) \mathbf{W}_{h_2} + \beta_{h_2}), \end{aligned} \quad (17)$$

where $\mathbf{H}_2$ is the output of the second feature map node obtained through a linear transformation from the first node. And $\mathbf{W}_{h_2}$ and β_{h_2} , like the above parameters, are randomly generated weight matrices and parameters used.

The subsequent feature mapping nodes follow a similar processing. Therefore, the cascading connection method of feature mapping nodes can be summarized as follows:

$$\mathbf{H}_k = \phi(\mathbf{H}_{k-1} \mathbf{W}_{h_k} + \beta_{h_k}) \quad (18)$$

where $\mathbf{W}_{h_k}$ and β_{h_k} are both randomly generated.

Next, the feature mapping nodes are connected to the enhancement nodes as follows:

$$\mathbf{Z}_j = \xi(\mathbf{H}^n \mathbf{W}_{z_j} + \beta_{z_j}), \quad (19)$$

where $\mathbf{Z}_j$ is the output of enhancement nodes obtained by non-linear transformation of features mapping nodes. ξ is the nonlinear transformation, and $\mathbf{W}_{z_j}$ and β_{z_j} are randomly generated parameters for enhancement nodes. $\mathbf{H}^n$ represents the entire output of the feature mapping node.

Therefore, we have obtained all the outputs of the feature mapping nodes, enhancement nodes, and image processing nodes. Next, we need to use the outputs of these three parts to obtain the final output of the BLN. The above process can be described as follows:

$$\begin{aligned} \mathbf{Y} &= [\Phi(\mathbf{X}; \mathbf{W}_h, \beta_h) | \Xi(\mathbf{H}^n; \mathbf{W}_z, \beta_z) | \Gamma(\mathbf{X}_{in}; \mathbf{W}_P, \beta_P)] \mathbf{W}_n^m \\ &= [\mathbf{H}_1, \dots, \mathbf{H}_n | \mathbf{Z}_1, \dots, \mathbf{Z}_m | \mathbf{P}_1, \dots, \mathbf{P}_p] \mathbf{W}_n^m \\ &= [\mathbf{H}^n | \mathbf{Z}^m | \mathbf{P}^p] \mathbf{W}_n^m, \end{aligned} \quad (20)$$

where $\mathbf{Y}$ is the final output of the BLN, which is the predicted network traffic type label, $\mathbf{W}_n^m$ is the weight matrix obtained from the outputs of features mapping nodes, enhancement nodes, and image processing nodes. Overall, this improved cascaded network can learn more effective features and process them quickly. In MTC problems, it has good performance.

3) *Cascade of Enhancement Nodes*: Similar to the cascade method of feature mapping nodes, the cascade connection of enhancement nodes is achieved through the same connection approach as the feature nodes mentioned earlier. However, with slight differences, enhancement nodes require the output of feature nodes as input. Therefore, feature nodes H_i can be described as follows:

$$\mathbf{H}_i = \phi(\mathbf{X} \mathbf{W}_{h_i} + \beta_{h_i}). \quad (21)$$

Next, the output of the feature nodes is subjected to nonlinear transformation processing to obtain the first group of enhanced nodes, which includes numerous individual enhanced nodes. The nonlinear transformation of feature nodes is as follows:

$$\mathbf{Z}_1 = \xi(\mathbf{H}^n \mathbf{W}_{z_1} + \beta_{z_1}) \quad (22)$$

where $\mathbf{Z}_1$ represents the nonlinear transformation of the features nodes $\mathbf{H}^n$, and $\mathbf{H}^n$ represents the entire output of the features mapping nodes. $\mathbf{W}_{z_1}$ and β_{z_1} are randomly generated weight matrices and coefficients used in transformations for enhancing nodes. Then, connecting the output of the first enhancement nodes to the next enhancement nodes, as described below:

$$\begin{aligned} \mathbf{Z}_2 &= \xi(\mathbf{Z}_1 \mathbf{W}_{z_2} + \beta_{z_2}) \\ &= \xi(\xi(\mathbf{H}^n \mathbf{W}_{z_1} + \beta_{z_1}) \mathbf{W}_{z_2} + \beta_{z_2}), \end{aligned} \quad (23)$$

where $\mathbf{Z}_2$ is the result obtained by non-linear transformation of the first enhancement nodes output. ξ is the nonlinear transformation function, and $\mathbf{W}_{z_2}$ and β_{z_2} are randomly generated parameters for the nonlinear transformation function.

The next enhancement nodes cascade connection method is similar to the above, which can be described as:

$$\mathbf{Z}_u = \xi^u(\mathbf{H}^n; \{\mathbf{W}_{z_i}, \beta_{z_i}\}_{i=1}^u) \quad (24)$$

However, it should be noted that the number of groups for enhancement nodes and the number of single group

enhancement nodes are two parameters, not the same. If it is the same parameter, the following situations will occur. When the variables are small, although the running speed is fast, the number of enhancement nodes is too small, and their nonlinear transformation cannot extract enough nonlinear features. When the variables are large, although there are enough enhancement nodes, the number of cascaded connection layers of enhancement nodes is too many, which will seriously consume training time. Therefore, we optimize the structure by setting the number of enhancement nodes in a single group to a larger value, which ensures that the nonlinear transformation can extract sufficient nonlinear features. Taking a smaller value for the number of groups of enhancement nodes can ensure fast training time and excellent network performance.

4) *Cascade of Features Mapping Nodes and Enhancement Nodes*: In this section, we introduce the cascading of features mapping nodes and enhancement nodes. Firstly, the feature mapping nodes are cascaded and connected, and then the connected feature nodes are linearly transformed and put into the enhancement nodes section. The enhancement nodes section also needs to be cascaded. When the features nodes output obtains the first group of enhancement nodes through nonlinear transformation, The output of the first group of enhancement nodes is transformed into the next group of enhancement nodes through their nonlinear transformation. By connecting in this way, the final group of enhancement nodes is obtained.

Next, we will introduce the cascaded connection between feature nodes and enhancement nodes. Firstly, the feature nodes are described as follows:

$$\mathbf{H}_k = \phi^k(\mathbf{X}; \{\mathbf{W}_{h_i}, \beta_{h_i}\}_{i=1}^k) \quad (25)$$

where $\mathbf{W}_{h_i}$ and β_{h_i} are randomly generated.

Next, the l groups of the enhancement nodes are generated by features mapping nodes, as follows:

$$\mathbf{Z}_l = \xi^l(\mathbf{H}^n; \{\mathbf{W}_{z_i}, \beta_{z_i}\}_{i=1}^l) \quad (26)$$

where $\mathbf{W}_{z_i}$ and β_{z_i} are random weights obtained through Gaussian distribution sampling.

After obtaining the output of the feature and enhancement nodes, we can obtain the final input of the broad learning network. The final input is as follows:

$$\mathbf{A}_{final} = [\mathbf{H}^n | \mathbf{Z}^m | \mathbf{I}^p] \quad (27)$$

The entire improved BLN can be described as follows:

$$\mathbf{Y} = [\mathbf{H}^n | \mathbf{Z}^m | \mathbf{I}^p] \mathbf{W}_n^m = \mathbf{A}_{final} \mathbf{W}_n^m. \quad (28)$$

C. Summary and Workflow

In this paper, we propose an improved algorithm IFCBLN based on BLN. This algorithm inherits the simple network architecture of BLN. It has few network layers and less weight coefficients than DL networks. This also determines that the proposed algorithm has a fast processing speed. At the same time, three network parts can extract feature information from network traffic data: features mapping nodes,

Algorithm 1: Workflow of the Proposed IFCBLN Method for MTC

Input: Malware traffic data for train and test $\hat{\mathbf{X}}_{train}$, $\hat{\mathbf{X}}_{test}$;
 Real traffic type labels for train $\mathbf{Y}_{train}$;
 Training hyperparameters Θ_{init} , Θ_{train} .

Output: Predicted labels results $\mathbf{Y}_{pre}$;

- 1 **[Stage of train]:**
- 2 Load $\hat{\mathbf{X}}_{train} \in \mathbb{R}^{a \times n}$, $\hat{\mathbf{Y}}_{train} \in \mathbb{R}^{a \times m}$;
- 3 Converting leader type data $\hat{\mathbf{X}}_{train}$ into image type data $\hat{\mathbf{X}}_{out}$;
- 4 Randomly generate $\mathbf{W}_h \in \mathbb{R}^{n' \times \tilde{l}}$, $\mathbf{W}_z \in \mathbb{R}^{(\tilde{l}l) \times \tilde{p}}$ and β_h, β_z ;
- 5 **if features mapping nodes then**
- 6 **if cascaded features mapping nodes then**
- 7 Compute nodes $\mathbf{H} \in \mathbb{R}^{a \times (\tilde{l}l)}$ by (16) ~ (18);
- 8 **else**
- 9 Compute nodes $\mathbf{H} \in \mathbb{R}^{a \times (\tilde{l}l)}$ by (5) and (6);
- 10 **end**
- 11 Linear transformation $\phi(\mathbf{W}_h, \mathbf{H}, \beta_h)$;
- 12 **end**
- 13 **if enhancement nodes then**
- 14 **if cascaded enhancement nodes then**
- 15 Compute nodes $\mathbf{Z} \in \mathbb{R}^{a \times (\tilde{q}q)}$ by (22) ~ (24);
- 16 **else**
- 17 Compute nodes $\mathbf{Z} \in \mathbb{R}^{a \times (\tilde{q}q)}$ by (7) and (8);
- 18 **end**
- 19 Non-linear transformation $\text{tansig}(\mathbf{W}_z, \beta_z, \mathbf{Z})$;
- 20 **end**
- 21 **if image processing nodes then**
- 22 Computer $\mathbf{X}_{in} \in \mathbb{R}^{a \times M}$ by (12);
- 23 Process $\mathbf{X}_{in}$ through $X_{in} = \frac{1}{\sqrt{M}} \sum f[n]\Phi[n]$;
- 24 Randomly generate $\mathbf{W}_{i_k} \in \mathbb{R}^{M \times \tilde{p}}$ and β_{i_k} ;
- 25 Compute nodes $\mathbf{I} \in \mathbb{R}^{a \times (\tilde{p}p)}$ by (14) and (15);
- 26 Non-linear transformation $\text{tansig}(\mathbf{W}_i, \beta_i, \mathbf{I})$;
- 27 **end**
- 28 Concatenate all nodes as input $\mathbf{A} = [\mathbf{H}|\mathbf{Z}|\mathbf{I}]$;
- 29 Equation $\mathbf{Y}_{train} = \mathbf{A}\mathbf{W}_{train}$, $\mathbf{A}^\dagger = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$;
- 30 Obtain equation solution $\mathbf{W}_{train} = \mathbf{A}^\dagger \mathbf{Y}_{train}$;
- 31 **[Stage of test]:**
- 32 Load $\hat{\mathbf{X}}_{test}$, $\mathbf{W}_h$, β_h , $\mathbf{W}_z$, β_z , $\mathbf{W}_i$, β_i , $\mathbf{Y}_{train}$;
- 33 Compute features mapping nodes $\mathbf{H}_{test}$, enhancement nodes $\mathbf{Z}_{test}$ and image processing nodes $\mathbf{I}_{test}$ through $\phi(\mathbf{W}_h, \mathbf{H}_{test}, \beta_h)$, $\text{tansig}((\mathbf{W}_i, \beta_i, \mathbf{I}_{test}), (\mathbf{W}_z, \beta_z, \mathbf{Z}_{test}))$;
- 34 Concatenate all nodes $\mathbf{A}_{test} = [\mathbf{H}_{test}|\mathbf{Z}_{test}|\mathbf{I}_{test}]$;
- 35 Get the classification labels by $\mathbf{Y}_{pre} = \mathbf{A}_{test} \mathbf{W}_{train}$;

enhancement nodes, and image processing nodes. These three nodes extract the features of network traffic data through linear transformation, nonlinear transformation, and image processing transformation methods, respectively. Therefore, it can be ensured that the proposed algorithm not only has fast training speed but also excellent performance. The workflow of IFCBLN can be found in **Algorithm 1**.

V. EXPERIMENT RESULTS AND ANALYSIS

This section mainly presents our proposed IFCBLN method and the algorithms for comparison. At the same time, it also introduces the dataset used, data preprocessing, related parameter settings, and experimental equipment used. When comparing with other algorithms, we compare them from various aspects such as computer overhead, test accuracy, precision, recall, and F1 scores.

A. Datasets, Data Preprocessing and Experiment Settings

1) *Datasets*: To test the performance of the model proposed in this article, we conduct experiments on two public network traffic datasets, including encrypted malicious traffic and normal traffic. Both datasets consist of 80% training dataset and 20% testing dataset. Next, we will introduce the dataset used in the experiment as follows:

- USTC-TFC2016 dataset [58] consists of real-world network traffic, including ten types of normal network traffic and ten types of malicious network traffic. This dataset is divided into 10 categories of normal traffic, 10 categories of malicious traffic, and 20 mixed categories of normal and malicious traffic. Correspondingly, there are 134563 samples, 138145 samples, and 272708 samples, respectively.
- ISCXVPN2016 dataset [55] is a dataset used for private networks to detect encrypted malicious traffic. The entire dataset is divided into network traffic under VPN and network traffic under non-VPN. This includes 12 types of encrypted network traffic. There are a total of 39446 samples in the entire dataset.

2) *Data Preprocessing*: To establish an effective training task interface, we first use the network packet analysis tool USTC-TK2016 [58] to convert the data into grayscale images, and extract and analyze network traffic features from the perspective of computer vision. The entire data preprocessing process can be divided into the following steps: firstly, we split the original traffic data into discrete data streams, and then anonymize the data packets by replacing MAC addresses, IP addresses, and port numbers. Secondly, to ensure a more reliable training process, duplicate or empty content must be removed. Trim all stream files to standardized 784 bytes to ensure consistency. Thirdly, converting to a grayscale image means proportionally transforming its numerical values into grayscale values ranging from 0 to 255 based on the maximum value. In this step, we convert the data stream file into a grayscale image for visualization. Each 784 bytes data stream is converted into a 28×28 pixel grayscale image. Finally, to speed up computer data retrieval, we save these images in IDX format as input data for the neural network. Through the above process, we successfully transform the original network traffic data into image format data, providing appropriate input for subsequent training tasks.

3) *Experiment Settings*: The experiments were conducted using PyTorch 2.0.1 and Python 3.8. In the proposed IFCBLN method, we need to manually set some parameters. After a series of tests, we found the optimal parameters that would make the model perform well. However, these parameters are

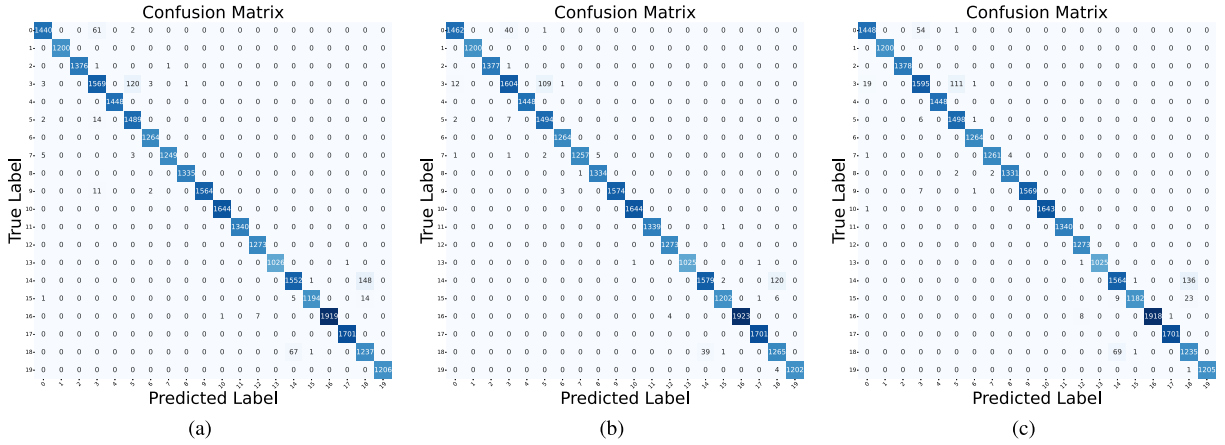


Fig. 5. The confusion matrix of the three cascaded methods on USTC-TFC2016 dataset.

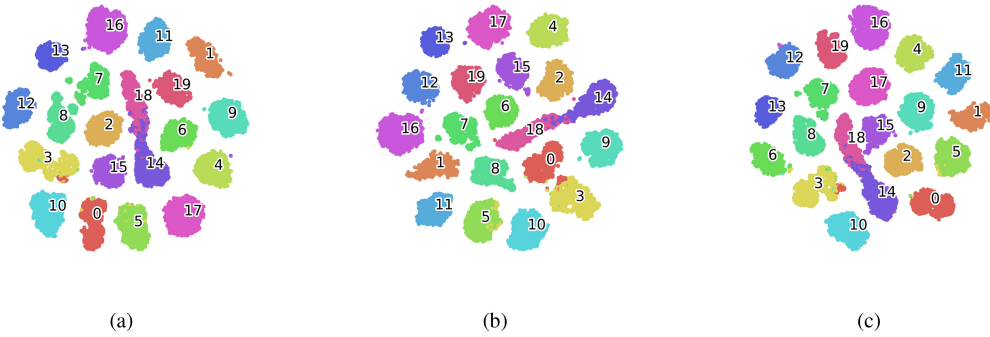


Fig. 6. The T-SNE of the three cascaded methods on the USTC-TFC2016 dataset.

TABLE I
PARAMETERS SETTING OF PROPOSED IFCBLN METHOD

Configuration	Values
Features Mapping Nodes	65×10
Enhancement Nodes	200×10
Groups of Enhancement Node	3
Image Processing Node	100
Split Ratio	8:2
Activation Function	tansig

not applicable to all scenarios, and in different scenarios, it is necessary to find another set of optimal parameters. The parameters of the proposed IFCBLN method are presented in Table I.

B. Classification Performance of IFCBLN

The proposed IFCBLN method was experimentally analyzed on two public datasets, evaluating the IFCBLN method based on three evaluation metrics: computational overhead, test accuracy, precision, recall, and F1 scores. The experimental results indicate that our proposed method performs well. We propose three image feature embedding cascaded broad learning networks: a cascade of features nodes (IFCFBLN), a cascade of enhancement nodes (IFCEBLN), and a cascade of features nodes and enhancement nodes (IFCFEBLN), and analyze them. In the USTC-TFC2016 dataset, IFCFBLN, IFCEBLN, and IFCFEBLN achieve accuracies of 99.16%,

99.383% and 99.543% for benign traffic in 10 categories, respectively. They achieve accuracies of 98.295%, 98.813% and 98.531% for malicious traffic in 10 categories. Among the 20 mixed categories of traffic, their accuracies are 97.912%, 98.513% and 98.321%. In the ISCXVPN2016 dataset, IFCFBLN, IFCEBLN, and IFCFEBLN achieve the accuracy of 90.93%, 90.887% and 90.824% in traffic of 12 types of VPN and Non-VPN. The above results are all shown in the Fig. 11. Due to the encrypted network traffic of the ISCXVPN2016 dataset, identification is difficult. Therefore, the classification results may be slightly worse than those of the USTC-TFC2016 dataset. This is a good result, far less than the computational overhead of deep learning.

In addition, to further demonstrate the classification performance of IFCBLN, we use the confusion matrix and T-SNE visualization tool to demonstrate the classification performance of the proposed method on two datasets. In Fig. 5, this is the confusion matrix for 20 mixed benign and malicious traffic in the USTC-TFC2016 dataset. Fig. 5(a), Fig. 5(b) and Fig. 5(c) are the confusion matrices of IFCFBLN, IFCEBLN and IFCFEBLN for 20 categories. In Fig. 6, the T-SNE of IFCFBLN, IFCEBLN, and IFCFEBLN show the classification results of visualization for 20 mixed benign and malicious traffic in the USTC-TFC2016 dataset. The confusion matrix of the ISCXVPN2016 dataset is shown in Fig. 7. Meanwhile, in Fig. 8, the T-SNE of IFCFBLN, IFCFEBLN, and IFCFEBLN show the classification results of visualization in the ISCXVPN2016 dataset. In the T-SNE

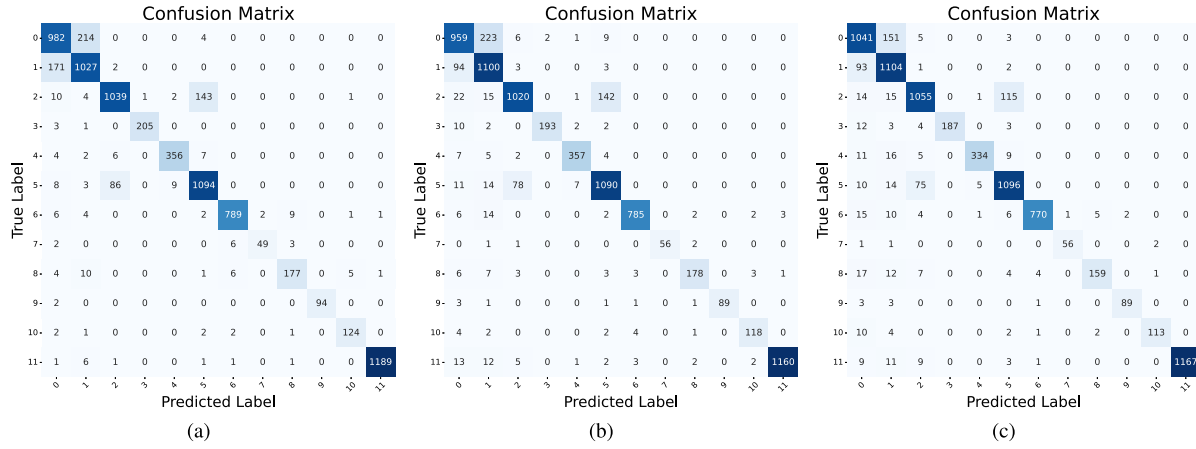


Fig. 7. The confusion matrix of the three cascaded methods on ISCXVPN2016 dataset.

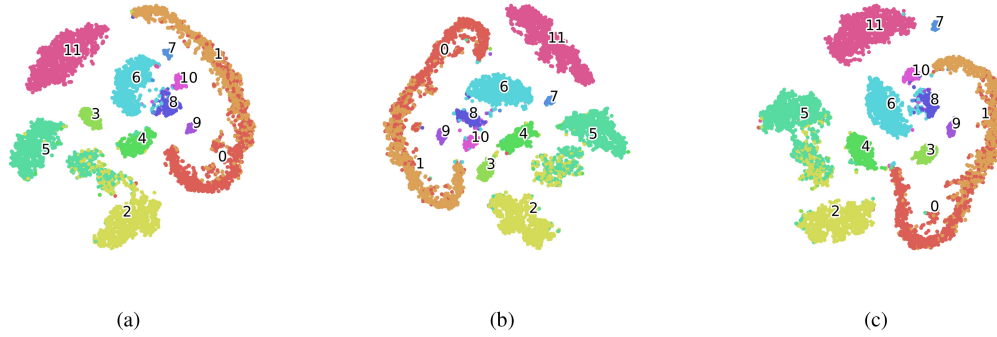


Fig. 8. The T-SNE of the three cascaded methods on the ISCXVPN2016 dataset.

analysis, there is partial overlap between Category 14 and Category 18 in the USTC-TFC2016 dataset, indicating that the traffic data of these two categories are quite similar. Similarly, in the ISCXVPN2016 dataset, multiple overlaps occur between Category 0 and Category 1, as well as between Category 2 and Category 5. This suggests a high similarity among these categories, making it challenging for the model to accurately classify and distinguish them.

C. Comparison With Other Methods

To comprehensively evaluate our proposed method, we compared IFCBLN with traditional ML algorithms and the latest DL algorithms. As follows:

- **SVM** and **Naive Bayes** algorithms are ML-based algorithms that use statistical features for network traffic classification.
- **CL-ETC** [24], **RVCNN** [25] and **CVCNN** [25] are DL based algorithms that are all based on CNN [56], [57] and improved upon. They have strong feature extraction capabilities in both CV and natural language processing (NLP) fields.

We use accuracy, precision, recall, and F1 scores to compare with the comparison algorithms. These indicators can be represented as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (29a)$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (29b)$$

$$Precision = \frac{TP}{TP + FP} \quad (29c)$$

$$Recall = \frac{TP}{TP + FN} \quad (29d)$$

$$F_1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (29e)$$

where TP, FP, TN and FN represent true positive, false positive, true negative and false negative, respectively. The proposed IFCBLN method was experimentally conducted on the CPU platform. SVM and Naive Bayes algorithms were also tested on CPU platforms. Due to its deep network structure and feedback mechanism, deep learning algorithms have excellent classification performance. But the disadvantage is that the training time is long. In Table II, the training time of all algorithms is shown. The training time of the CL-ETC algorithm is significantly longer than that of the algorithm proposed in this paper, and its classification accuracy is not as high as the proposed algorithm on both datasets. The RVCNN and CVCNN algorithms consist of 8 convolutional layers and 9 convolutional layers, respectively. This means that its training time is greatly increased, and the fact is the same. Their training time is much longer than that of IFCBLN. The algorithms proposed in this article are IFCBLN, IFCEBLN, and IFCFEBLN. They only account for 0.26%, 0.55%, and 0.48% of the training time of CVCNN, respectively, in the 20 classification tasks of the USTC-TFC2016 dataset. Among

TABLE II
COMPUTATIONAL OVERHEAD OF IFCBLN AND COMPARED METHODS

Methods	USTC-TFC2016 dataset						ISCXVPN2016 dataset	
	20 classes		10 classes(Benign)		10 classes(Malware)		train time	test time
	train time	test time	train time	test time	train time	test time		
SVM	2925.4	2450.33	1406.69	605.66	690.172	442.52	153.99	91.62
Naive Bayes	0.384	0.054	0.172	0.024	0.174	0.026	0.069	0.011
CL-ETC	9908.72	1.651	4901.31	0.817	5010.129	0.839	2488.93	0.512
RVCNN	17187.787	3.801	8479.685	1.852	8672.754	1.905	2472.522	0.554
CVCNN	44289.225	7.611	21508.65	3.612	27159.13	3.834	7499.9	1.068
IFCFBLN (proposed)	114.682	13.616	59.788	7.144	61.235	6.969	30.625	2.649
IFCEBLN (proposed)	245.12	25.207	131.134	12.438	131.779	12.07	44.434	3.558
IFCFEBLN (proposed)	214.682	22.358	108.34	11.506	111.447	10.921	115.617	6.525

the 10 benign traffic classification tasks, they only account for 0.28%, 0.61%, and 0.5% of the training time of CVCNN. Among the 10 malicious traffic classification tasks, they only account for 0.23%, 0.48%, and 0.41% of the training time of CVCNN. In the ISCXVPN2016 dataset, IFCFBLN, IFCEBLN, and IFCFEBLN account for 0.41%, 0.6%, and 1.54% of the training time of CVCNN. In the USTC-TFC2016 dataset, parameter configurations for IFCFBLN and IFCEBLN models are largely consistent. Specifically, the cascade of feature mapping nodes, which involves minimal parameters, has negligible impact on runtime. However, the enhancement nodes contain more parameters, contributing significantly to computational time. Variations in CPU usage during experiments caused fluctuations of 10-30 seconds in training time between trials, affecting consistency between the two models. To address this, additional experiments were conducted under stable CPU conditions, revealing that the training times for IFCFBLN and IFCEBLN were effectively identical, as expected, given their similar parameter configurations. Moreover, the cascading of feature mapping nodes preserves the weights from preceding nodes, reducing computational overhead.

To intuitively demonstrate the time advantage of the proposed algorithms, we use horizontal bar charts to describe the training and testing time in the USTC-TFC2016 and ISCXVPN2016 datasets. We use a logarithmic scale, which can better reflect the data. Among them, the training time is shown in Fig. 9, and the testing time is shown in Fig. 10. From them, it can be seen that compared to deep learning algorithms, the proposed broad learning algorithms have significant advantages.

The accuracy results of IFCBLN and comparison algorithms on the USTC-TFC2016 and ISCXVPN2016 datasets are shown in Fig. 11. The accuracy of IFCFBLN, IFCEBLN, and IFCFEBLN is significantly higher than SVM and Naive Bayes. Although Naive Bayes has low computational overhead, its classification accuracy is significantly poor. Compared with RVCNN and CVCNN, our method has an accuracy difference of within 1% in the 10 benign traffic classification tasks in the USTC-TFC2016 dataset. In the ISCXVPN2016 dataset, IFCFBLN, IFCEBLN, and IFCFEBLN achieved accuracies of 90.93%, 90.887%, and 90.824%, respectively. Better than the 89.21% accuracy of CVCNN. Slightly inferior to the 92.89% accuracy of RVCNN. In addition to testing accuracy,

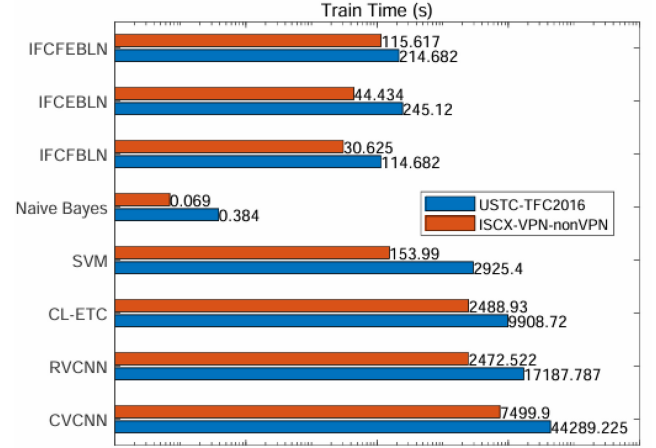


Fig. 9. Train time of all algorithms in USTC-TFC2016 and ISCXVPN2016 datasets.

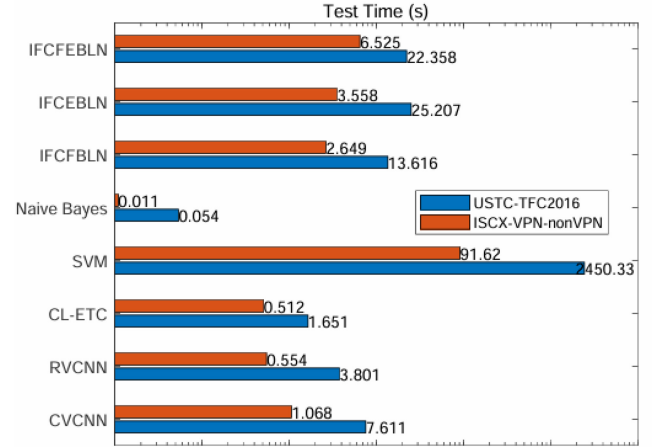


Fig. 10. Test time of all algorithms in USTC-TFC2016 and ISCXVPN2016 datasets.

we also evaluate the proposed IFCBLN using three metrics: precision (PC), recall (RC) and f1 score (F1). The PR, RC, and F1 of the proposed algorithm and the comparison algorithms are presented in Table IV on the USTC-TFC2016 dataset. The PR, RC and F1 of the ISCXVPN2016 dataset are shown in Table III. Especially in the ISCXVPN2016 dataset, our proposed IFCBLN algorithm outperforms all five

TABLE III
ABLATION STUDY OF KEY COMPONENTS IN IFCBLN ON ISCXVPN2016

Methods	ISCXVPN2016 dataset		
	PR	RC	F1
SVM	91.87%	89.965%	90.76%
Naive Bayes	59.185%	59.976%	56.075%
CL-ETC	57.15%	68.62%	58.89%
RVCNN	85.81%	86.37%	85.75%
CVCNN	84.49%	85.92%	85.05%
IFCFBLN (proposed)	93.728%	92.359%	92.992%
IFCEBLN (proposed)	93.667%	91.337%	92.401%
IFCFEBLN (proposed)	94.321%	89.747%	91.832%

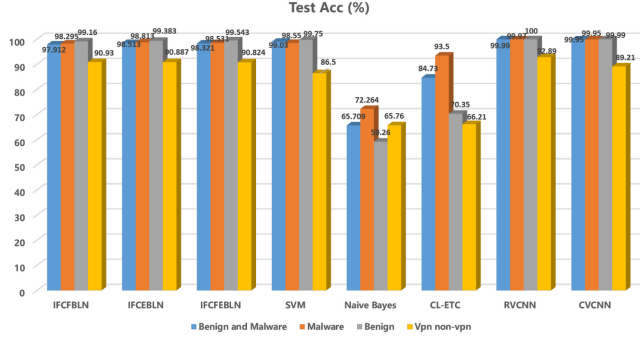


Fig. 11. Performance of IFCBLN and compared methods.

algorithms in terms of three metrics. However, the IFCFEBLN method, with its more complex parameters, risks overfitting by capturing noise and specific patterns in the training data. This leads to solid performance on the training set but reduced generalization on the test set, weakening the ability of the model to identify certain class samples and lowering recall. Additionally, complex models are prone to finding local optima in high-dimensional spaces, which can increase class confusion, causing samples from smaller classes to be misclassified as larger ones, thereby raising false negatives and further reducing recall. For example, in tasks involving malicious traffic or radiation source detection, similar patterns across certain categories may lead to misclassification. Furthermore, class imbalance is more pronounced in complex models, where the model may prioritize majority classes to improve accuracy while neglecting minority classes, further diminishing recall for underrepresented categories. Meanwhile, we compare the computational complexity and parameters of the DL model with the proposed methods.

The computational complexity and parameter count of the proposed algorithm and deep learning model are presented in Table V. The testing time for our algorithm is slower than other models because deep learning models typically run on GPUs, while ours runs on CPUs. Although training on CPUs offers some advantages, our approach faces drawbacks during testing due to the GPU-optimized operations of PyTorch and the speed of GPU-based matrix computations. We have now deployed testing on GPUs using the ISCXVPN2016 dataset, reducing the testing time to about 1 second. However, since some modules remain on CPUs, further optimization is still possible.

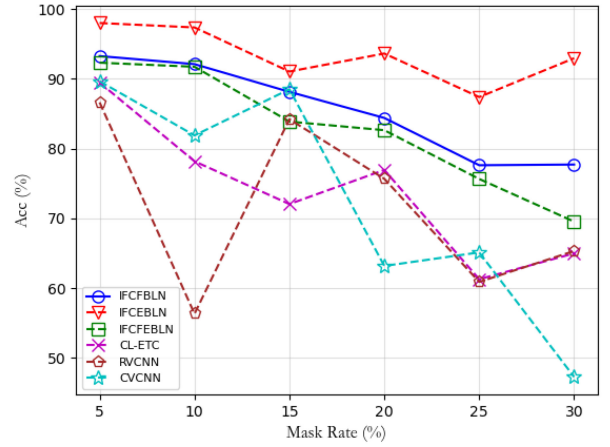


Fig. 12. Results of mask testing in USTC-TFC2016 dataset.

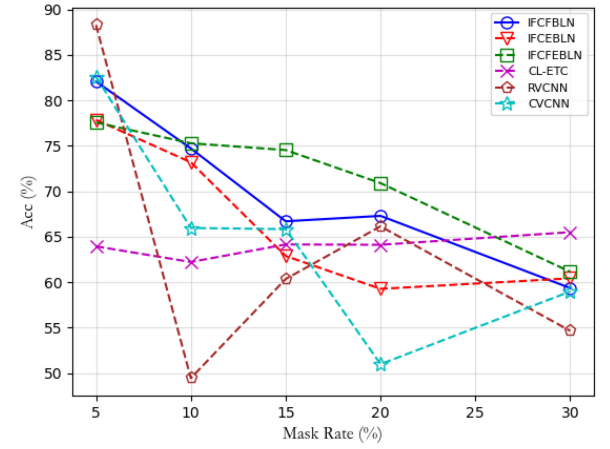


Fig. 13. Results of mask testing in ISCXVPN2016 dataset.

D. Mask Testing

Due to the actual network environment and various encryption methods, the network data used is not complete. This is a challenge for proposed algorithm models. Therefore, based on the above actual situation, we use mask testing to simulate the actual network environment. In the USTC-TFC2016 dataset, we masked 5%, 10%, 15%, 20%, 25%, and 30% of the test data to complete the masking test. Due to the inclusion of mask testing, this poses a challenge to the generalization ability of the algorithm model. The generalization ability of ML is far inferior to DL, so the proposed method is only compared with three DL algorithms: CL-ETC, RVCNN, and CVCNN. The results show in Fig. 12 that the accuracy of IFCFBLN and IFCEBLN is all due to the comparison of the three algorithms under different scale masks. The accuracy of IFCFEBLN is inferior to that of CVCNN at 15%, and it is also better than the three algorithms compared in other cases. In addition, in the ISCXVPN2016 dataset, we masked 5%, 10%, 15%, 20%, and 30% of the test data. Proposed algorithms also performed well, and the results are shown in Fig. 13.

TABLE IV
ABLATION STUDY OF KEY COMPONENTS IN IFCBLN ON USTC-TFC2016

Methods	USTC-TFC2016 dataset								
	20 classes			10 classes(Benign)			10 classes(Malware)		
	PR	RC	F1	PR	RC	F1	PR	RC	F1
SVM	99.12%	99.11%	99.11%	99.786%	99.801%	99.793%	98.649%	98.623%	98.636%
Naive Bayes	67.554%	67.855%	63.235%	61.182%	62.675%	54.745%	75.127%	73.333%	72.169%
CL-ETC	85.49%	85.6%	83.5%	80.45%	71.07%	71.54%	93.48%	93.67%	93.36%
RVCNN	99.98%	99.98%	99.98%	100%	100%	100%	99.97%	99.96%	99.97%
CVCNN	99.96%	99.95%	99.95%	99.99%	99.99%	99.99%	99.95%	99.95%	99.95%
IFCFBLN (proposed)	98.113%	98.14%	98.11%	99.27%	99.326%	99.286%	98.394%	98.396%	98.391%
IFCEBLN (proposed)	98.645%	98.677%	98.65%	99.463%	99.506%	99.478%	98.876%	98.874%	98.873%
IFCFEBLN (proposed)	98.487%	98.507%	98.486%	99.596%	99.633%	99.611%	98.616%	98.596%	98.603%

TABLE V
COMPUTATIONAL COMPLEXITY AND PARAMETERS OF DL AND THE PROPOSED METHODS

Methods	params($\times 10^6$)	FLOPs(MB)
CVCNN	0.32	1171.64
RVCNN	0.179	311.69
CL-ETC	1.123	463.88
IFCFBLN (proposed)	3.192	117.60
IFCEBLN (proposed)	4.494	322.01
IFCFEBLN (proposed)	4.494	254.33

VI. CONCLUSION

This paper proposed the IFCBLN method for MTC problems, which can be quickly trained on CPU platforms. The proposed method, based on the basic BLN, solved the problem of identifying encrypted malicious traffic by cascading feature mapping nodes and enhancement nodes in multiple ways, as well as adding image processing nodes. The proposed methods achieved good results. This algorithm was compared with classical ML algorithms and advanced DL, and compared through multiple metrics. This further demonstrated the excellent performance of IFCBLN. It also eliminated the limitations of GPU, making it more convenient to deploy on MTC devices. The experimental results show that our proposed method performed well. In the subsequent process, we would further improve the connection method of the network model to accelerate the model's training time. We would also search for stronger feature extraction methods as better incremental nodes.

REFERENCES

- [1] Y. Zhang, G. Gui, and S. Mao, "A lightweight malware traffic classification method based on a broad learning architecture," *IEEE Internet Things J.*, vol. 10, no. 23, pp. 21131–21132, Dec. 2023.
- [2] S. Xu, G. Geng, X. Jin, D. Liu, and J. Weng, "Seeing traffic paths: Encrypted traffic classification with path signature features," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2166–2181, 2022.
- [3] P. Lin, K. Ye, Y. Hu, Y. Lin, and C. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.
- [4] X. Xiao, W. Xiao, R. Li, X. Luo, H. Zheng, and S. Xia, "EBSNN: Extended byte segment neural network for network traffic classification," *IEEE Trans. Depend. Secure Comput.*, vol. 19, no. 5, pp. 3521–3538, Sep./Oct. 2022.
- [5] G. Gui, M. Liu, F. Tang, N. Kato, and F. Adachi, "6G: Opening new horizons for integration of comfort, security, and intelligence," *IEEE Wireless Commun.*, vol. 27, no. 5, pp. 126–132, Oct. 2020.
- [6] C. Lee, H. Nam, D. Kim, S. Kim, and D. Lee, "A D-band variable gain low noise amplifier in a 28-nm CMOS process for 6G wireless communications," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 131–135, Jan. 2024.
- [7] H. Qi, J. Wang, W. Li, Y. Wang, and T. Qiu, "A blockchain-driven IIoT traffic classification service for edge computing," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2124–2134, Feb. 2021.
- [8] B. Kang, D. Kim, and H. Choo, "Internet of Everything: A large-scale autonomic IoT gateway," *IEEE Trans. Multi-Scale Comput. Syst.*, vol. 3, no. 3, pp. 206–214, Jul. 2017.
- [9] J. Li, W. Liang, W. Xu, Z. Xu, Y. Li, and X. Jia, "Service home identification of multiple-source IoT applications in edge computing," *IEEE Trans. Serv. Comput.*, vol. 16, no. 2, pp. 1417–1430, Mar./Apr. 2023.
- [10] S. Pokhrel, H. Vu, and A. Cricenti, "Adaptive admission control for IoT applications in home Wi-Fi networks," *IEEE Trans. Mob. Comput.*, vol. 19, no. 12, pp. 2731–2742, Dec. 2020.
- [11] Z. Li, T. Sen, H. Shen, and M. Chuah, "A study on the impact of memory DoS attacks on cloud applications and exploring real-time detection schemes," *IEEE/ACM Trans. Netw.*, vol. 30, no. 4, pp. 1644–1658, Aug. 2022.
- [12] M. Al-Hisnawi and M. Ahmadi, "Deep packet inspection using quotient filter," *IEEE Commun. Lett.*, vol. 20, no. 11, pp. 2217–2220, Nov. 2016.
- [13] R. Zhao et al., "A novel self-supervised framework based on masked autoencoder for traffic classification," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2012–2025, Jun. 2024.
- [14] C. Fu, Q. Li, M. Shen, and K. Xu, "Frequency domain feature based robust malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 452–467, Feb. 2023.
- [15] S. Zhu, X. Xu, J. Zhao, and F. Xiao, "LKD-STNN: A lightweight malicious traffic detection method for Internet of Things based on knowledge distillation," *IEEE Internet Things J.*, vol. 11, no. 4, pp. 6438–6453, Feb. 2024.
- [16] Z. Xu, X. Wang, and Y. Zhang, "Towards persistent detection of DDOS attacks in NDN: A sketch-based approach," *IEEE Trans. Depend. Secure Comput.*, vol. 20, no. 4, pp. 3449–3465, Jul./Aug. 2023.
- [17] S. Cui, C. Dong, M. Shen, Y. Liu, B. Jiang, and Z. Lu, "CBSeq: A channel-level behavior sequence for encrypted malware traffic detection," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5011–5025, 2023.
- [18] Y. Shi, X. Zhang, Z. He, and J. Yang, "A novel malware traffic classification method based on differentiable architecture search," in *Proc. IEEE 96th Veh. Technol. Conf. (VTC-Fall)*, London, U.K., Sep. 2022, pp. 1–5.
- [19] A. Sadeghzadeh, S. Shiravi, and R. Jalili, "Adversarial network traffic: Towards evaluating the robustness of deep-learning-based network traffic classification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1962–1976, Jun. 2021.
- [20] J. Zhang, F. Li, and F. Ye, "Sustaining the high performance of AI-based network traffic classification models," *IEEE/ACM Trans. Netw.*, vol. 31, no. 2, pp. 816–827, Apr. 2023.
- [21] H. Yao, C. Liu, P. Zhang, S. Wu, C. Jiang, and S. Yu, "Identification of encrypted traffic through attention mechanism based long short term memory," *IEEE Trans. Big Data.*, vol. 8, no. 1, pp. 241–252, Feb. 2022.

- [22] Y. Lin, M. Wang, X. Zhou, G. Ding, and S. Mao, "Dynamic spectrum interaction of UAV flight formation communication with priority: A deep reinforcement learning approach," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 3, pp. 892–903, Sep. 2020.
- [23] Y. Lin, Y. Tu, Z. Dou, L. Chen, and S. Mao, "Contour stella image and deep learning for signal recognition in the physical layer," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 1, pp. 34–46, Mar. 2021.
- [24] Z. Zhao, Y. Guo, J. Wang, H. Wang, C. Zhang, and C. An, "CL-ETC: A contrastive learning method for encrypted traffic classification," in *Proc. IFIP Netw. Conf.*, Catania, Italy, 2022, pp. 1–9.
- [25] Y. Wang, G. Gui, H. Gacanin, T. Ohtsuki, O. Dobre, and H. Poor, "An efficient specific emitter identification method based on complex-valued neural networks and network compression," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2305–2317, Aug. 2021.
- [26] C. Chen and Z. Liu, "Broad learning system: An effective and efficient incremental learning system without the need for deep architecture," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 1, pp. 10–24, Jan. 2018.
- [27] C. Chen, Z. Liu, and S. Feng, "Universal approximation capability of broad learning system and its structural variations," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 4, pp. 1191–1204, Apr. 2019.
- [28] S. Feng and C. Chen, "Fuzzy broad learning system: A novel neuro-fuzzy model for regression and classification," *IEEE Trans. Cybern.*, vol. 50, no. 2, pp. 414–424, Feb. 2020.
- [29] X. Gong, T. Zhang, C. Chen, and Z. Liu, "Research review for broad learning system: Algorithms, theory, and applications," *IEEE Trans. Cybern.*, vol. 52, no. 9, pp. 8922–8950, Sep. 2022.
- [30] H. Zhao, J. Zheng, W. Deng, and Y. Song, "Semi-supervised broad learning system based on manifold regularization and broad network," *IEEE Trans. Circuits Syst. I, Reg. Pap.*, vol. 67, no. 3, pp. 983–994, Mar. 2020.
- [31] X. Yuan, J. Chen, N. Zhang, X. Fang, and D. Liu, "A federated bidirectional connection broad learning scheme for secure data sharing in Internet of Vehicles," *China Commun.*, vol. 18, no. 7, pp. 117–133, Jul. 2021.
- [32] X. Wang, Y. Zhu, S. Han, L. Yang, H. Gu, and F. Wang, "Fast and progressive misbehavior detection in Internet of Vehicles based on broad learning and incremental learning systems," *IEEE Internet Things J.*, vol. 9, no. 6, pp. 4788–4798, Mar. 2022.
- [33] S. Han, K. Zhu, M. Zhou, and X. Liu, "Evolutionary weighted broad learning and its application to fault diagnosis in self-organizing cellular networks," *IEEE Trans. Cybern.*, vol. 53, no. 5, pp. 3035–3047, May 2023.
- [34] Y. Zhang, Y. Peng, J. Sun, G. Gui, Y. Lin, and S. Mao, "GPU-free specific emitter identification using signal feature embedded broad learning," *IEEE Internet Things J.*, vol. 10, no. 14, pp. 13028–13039, Jul. 2023.
- [35] G. Liu, W. Shen, L. Gao, and A. Kusiak, "Active broad-transfer learning algorithm for class-imbalanced fault diagnosis," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–16, Jan. 2023.
- [36] P. Li, B. Sheng, and C. Chen, "Face sketch synthesis using regularized broad learning system," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 10, pp. 5346–5360, Oct. 2022.
- [37] Y. Chen, Q. Zhang, H. Jiang, Y. Luo, and Y. Chen, "A cognitive feature extracting method for space target," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Beijing, China, 2016, pp. 3148–3151.
- [38] Y. Lin, Y. Tu, and Z. Dou, "An improved neural network pruning technology for automatic modulation classification in edge devices," *IEEE Trans. Veh. Technol.*, vol. 69, no. 5, pp. 5703–5706, May 2020.
- [39] Y. Tu, Y. Lin, J. Wang, and J. U. Kim, "Semi-supervised learning with generative adversarial networks on digital signal modulation classification," *Comput., Mater. Continua*, vol. 55, no. 2, pp. 243–254, Feb. 2018.
- [40] B. Yamansavascilar, M. A. Guvensan, A. G. Yavuz, and M. E. Karsligil, "Application identification via network traffic classification," in *Proc. Int. Conf. Comput. Netw. Commun. (ICNC)*, Jan. 2017, pp. 843–848.
- [41] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779–796, Nov. 2019.
- [42] S. Chebrolu, A. Abraham, and J. P. Thomas, "Feature deduction and ensemble design of intrusion detection systems," *Comput. Security*, vol. 24, no. 4, pp. 295–307, 2005.
- [43] S. Datta, A. Kotha, U. Venkanna, and K. M. Rao, "XNetIoT: An extreme quantized neural network architecture for IoT environment using P4," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 5, pp. 5756–5767, Oct. 2024.
- [44] S. Jorgensen, J. Holodnak, J. Dempsey, K. Souza, A. Raghunath, and V. Rivet, "Extensible machine learning for encrypted network traffic application labeling via uncertainty quantification," *IEEE Trans. Artif. Intell.*, vol. 5, no. 1, pp. 420–433, Jan. 2024.
- [45] J. Yang, X. Jiang, Y. Lei, W. Liang, Z. Ma, and S. Li, "MTSecurity: Privacy-preserving malicious traffic classification using graph neural network and transformer," *IEEE Trans. Netw. Services Manag.*, vol. 21, no. 3, pp. 3583–3597, Jun. 2024.
- [46] E. Horowicz, T. Shapira, and Y. Shavitt, "Self-supervised traffic classification: Flow embedding and few-shot solutions," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 3, pp. 3054–3067, Jun. 2024.
- [47] K. Xu, X. Zhang, Y. Wang, T. Ohtsuki, B. Adebisi, and H. Sari, "Self-supervised learning malware traffic classification based on masked autoencoder," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17330–17340, May 2024.
- [48] X. Zhang, Q. Wang, M. Qin, Y. Wang, T. Ohtsuki, and B. Adebisi, "Enhanced few-shot malware traffic classification via integrating knowledge transfer with neural architecture search," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5245–5256, 2024.
- [49] X. Xu et al., "Advancing malware detection in network traffic with self-paced class incremental learning," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21816–21826, Jun. 2024.
- [50] W. Ge, Z. Cui, J. Wang, B. Tang, and X. Li, "MetaCluster: A universal interpretable classification framework for cybersecurity," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 3829–3843, 2024.
- [51] C. Sheng, Y. Yao, L. Zhao, P. Zeng, and J. Zhao, "Scanner-hunter: An effective ICS scanning group identification system," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 3077–3092, 2024.
- [52] X. Zhang, L. Hao, G. Gui, Y. Wang, B. Adebisi, and H. Sari, "An automatic and efficient malware traffic classification method for secure Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 8448–8458, Mar. 2024.
- [53] M. He, Y. Huang, X. Wang, P. Wei, and X. Wang, "A lightweight and efficient IoT intrusion detection method based on feature grouping," *IEEE Internet Things J.*, vol. 11, no. 2, pp. 2935–2949, Jan. 2024.
- [54] M. Lin, K. Yang, Z. Yu, Y. Shi, and C. Chen, "Hybrid ensemble broad learning system for network intrusion detection," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, pp. 5622–5633, Apr. 2024.
- [55] G. Draper-Gil, A. Lashkari, M. Mamun, and A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Security Privacy*, 2016, pp. 407–414.
- [56] Y. Tu, Y. Lin, C. Hou, and S. Mao, "Complex-valued networks for automatic modulation classification," *IEEE Trans. Veh. Technol.*, vol. 69, no. 9, pp. 10085–10089, Sep. 2020.
- [57] Y. Lin, H. Zhao, X. Ma, Y. Tu, and M. Wang, "Adversarial attacks in modulation recognition with convolutional neural networks," *IEEE Trans. Rel.*, vol. 70, no. 1, pp. 389–401, Jan. 2020.
- [58] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, 2017, pp. 712–717.